GenIA: Generative Index Advisor for Dynamic Workloads and Data

Xian Lyu, Chen Lin, *Member, IEEE*, Yihang Zheng, Zhifeng Bao, Yiming Zhang, and Guoliang Li, *Fellow, IEEE*

***Abstract*—An ideal index advisor needs to effectively manage changes in workload and data, but current approaches fall short in both effectiveness and efficiency because of intrinsic limitations in their frameworks. Heuristic-based methods struggle with efficiency due to their rigid algorithms and lack of adaptive learning capabilities. Reinforcement learning techniques often fail to consistently reach an optimal policy. Classification methods require vast amounts of labeled workloads that include optimal indexes. Additionally, none of the learning-based strategies are equipped to handle shifts in data. To overcome these limitations, this paper presents a new index advisor for dynamic workloads and data, `GenIA`, which learns to generate a sequence of the recommended index configuration based on historical experience. The generative framework of `GenIA` avoids erroneous trials to explore bad actions and reliance on high-quality positive and negative examples. Specifically, its novelty exhibits in three aspects. (1) `GenIA` is empowered with novel attention mechanisms to capture implicit relationships between indexable columns. (2) `GenIA` combines comprehensive features extracted from workloads, data manipulation statements, and underlying data to effectively capture workload shifts and subtle data shifts. (3) `GenIA` adopts a novel perturbation-based training strategy to enhance the diversity of training samples and to improve the model parameters' robustness. Extensive experiments on various benchmarks under varying levels of workload and data shifts demonstrate that `GenIA` outperforms SOTA heuristic-based IA Extend on average by about 7.5%, while utilizing less than 1% of the inference time, and surpasses SOTA learning-based IA SWIRL by 25% − 30% in scenarios with significant workload and data shifts.**

***Index Terms*—Database management, query processing, artificial intelligence.**

I. Introduction

AS ONE of the most important database techniques for optimizing workload performance, index selection must be both *efficient* (i.e., with a small overhead to decide the selection) and *effective* (i.e., reduce query latency with the selected indexes) [1]. It becomes even more challenging when facing the dynamic nature of workloads and data. Fig. 1(a) illustrates an example of *dynamic workloads:* consider BusTracker [2], a mobile application for real-time tracking of public transit systems. During working hours (6AM-6PM), queries are predominantly commute-related and frequently need to access columns like `agency_id, start_date`. Conversely, during after-work hours (6PM-8PM), increased operation-related queries accessing columns like `user_id, route_id` are witnessed. Fig. 1(b) illustrates an instance of *dynamic data:* CH-BenCHmark [3], a benchmark for Hybrid Transactional and Analytical Processing workloads. After "`delivery`" transactions cause large deletions in the "`new_order`" table, the original index on columns of the table "`new_order`" is no longer a good choice.

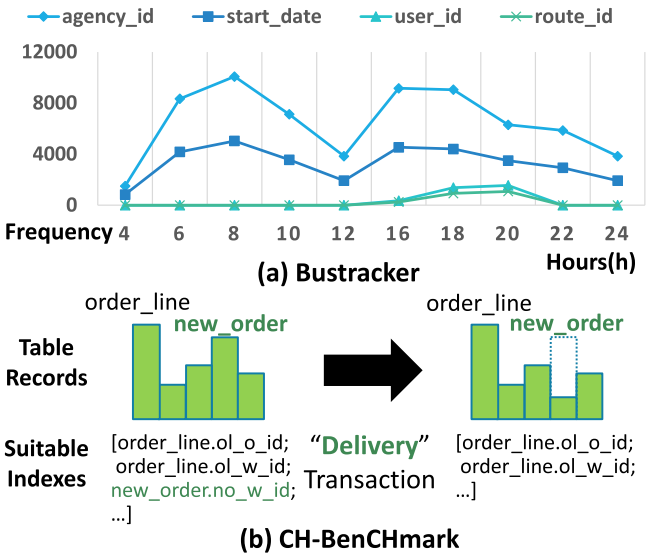

Fig. 1. Examples of dynamic workload/data. (a) Workloads: Various columns at different times throughout the day. (b) Transactional workloads alter data distributions.

We call an index selection method as an index advisor (IA). Most existing IAs [1], [4], [5], [6], [7] sacrifice efficiency for effectiveness, especially under dynamic workloads and data. When the workload changes, *heuristic-based IAs* have to re-run the algorithm, and most *learning-based IAs* have to re-train

Received 9 March 2025; revised 8 May 2026; accepted 28 May 2026. Date of publication 1 June 2026; date of current version 8 August 2026. The work of Chen Lin was supported in part by the Natural Science Foundation of China under Grant 62432011 and in part by the Fujian Provincial Natural Science Foundation of China under Grant 2025J010001. Recommended for acceptance by F. Banaei-Kashani. *(Corresponding author: Chen Lin.)*

Xian Lyu, Chen Lin, and Yihang Zheng are with the School of Informatics, Xiamen University, Xiamen 361102, China (e-mail: xianlyu2023@stu.xmu.edu.cn; chenlin@xmu.edu.cn; yihang@stu.xmu.edu.cn).

Zhifeng Bao is with the School of EECS, The University of Queensland, Brisbane, QLD 4072, Australia (e-mail: zhifeng.bao@uq.edu.au).

Yiming Zhang is with the Institute of Intelligent Software and Systems, School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: zhangyiming@cs.sjtu.edu.cn).

Guoliang Li is with the Department of Computer Science, Tsinghua University, Beijing 100084, China (e-mail: liguoliang@tsinghua.edu.cn).

Digital Object Identifier 10.1109/TKDE.2026.3698793

the model. Although some recent studies [8], [9], [10] attempt to *directly recommend* indexes without expensive retraining or model updating for workload shifts, their effectiveness is *limited to replacing a small proportion of the workload templates*, as evidenced in our experiments. Even worse, to the best of our knowledge, all existing learning-based IAs are unable to recommend indexes directly under data shifts.

To summarize, three challenges must be overcome to maintain high efficiency and effectiveness of index advisors.

C1: How to *model* index selection under workload and data shifts. Existing IAs are commonly based on *index candidates*. On the one hand, existing IAs build a unique search space of index candidates for each workload and data, which may cause re-initiating the IA under large workload and data shifts. On the other hand, multi-column and single-column index candidates are treated individually (e.g. different states), compromising the IA's effectiveness [11], [12]. For example, since a large action space leads to the inherent convergence problem in reinforcement learning (RL), i.e., not reliably converging to an optimal policy, most RL-based IAs [7], [8], [9], [10] need to keep hundreds of trials and model copies to preserve indexing effectiveness. Furthermore, due to the NP-hard nature of the index selection problem, optimal index labels are too costly to collect to learn a complicated discriminative boundary. Thus, classification-based IAs [13] are also not suitable.

C2: How to extract representative *features* of workload and data. Previous studies [7], [8], [9], [10] focus on representing analytical workloads but overlook transactional workloads. Moreover, data features are completely ignored. Thus, they cannot learn the mapping function among optimal indexes, workloads, and data. It is non-trivial to extract transactional workload and data features. On the one hand, transactional workloads impact index selection, e.g., index maintenance costs brought by data manipulation statements. Workload features must reflect such an impact. On the other hand, data shifts, including changes in data volume and distribution, would also affect index selection, and they should be taken into account as part of data features.

C3: How to improve the IA's *generalization* ability during training. Existing studies is not able to generalize to diverse workloads and data. For example, SWIRL [10] is the SOTA learning-based IA that can generalize to workloads with unseen queries. Nevertheless, as shown in our experiments (cf., Section V-D), as the proportion of unknown queries increases to 63%, the effectiveness of SWIRL's recommended indexes drops by 36%. Since the training data that can be collected is inevitably limited, the generalization ability is a key issue that enables the IA to make direct recommendations and avoid expensive re-training and model updating.

Our Contributions. We propose `GenIA` (Generative Index Advisor), which can directly predict effective indexes under dynamic workloads and data.

Firstly, `GenIA` addresses C1 by *learning to generate* a sequence of tokens (including indexable columns, segmentation tokens that indicate multi-column indexes and other special tokens) for each workload and data. The vocabulary of tokens is universal for all workloads and data, allowing *direct index recommendation* without re-initializing the search space. Multi-column and single-column indexes are naturally intertwined through the generation of tokens, boosting the learning effectiveness of different index candidates. Since `GenIA` is supervised from historical index creation experience available in real-production systems, `GenIA` enjoys the advantage of supervised methods over RL methods, i.e., there is no need for erroneous trials to explore bad actions. Unlike classification methods [13], `GenIA` learns from workloads, data, and their appropriate (not necessarily optimal) index configurations and does not rely on many high-quality positive and negative examples. Furthermore, `GenIA` is empowered with novel attention mechanisms to more effectively capture the implicit relationships between indexable columns. (Section II).

Secondly, `GenIA` proposes a novel feature extraction method to extract workload and data features for each indexable column. The workload features include key operators extracted from the workloads' query plans and estimated index maintenance costs from data manipulation statements. The data features capture subtle changes in data volume and data distribution from a calculated data histogram for each indexable column. The workload and data features are combined to reflect the impacts of indexable columns regarding workload shifts and data changes. (Section III).

Thirdly, `GenIA` proposes a novel perturbation-based training strategy to improve its generalization ability effectively. At the training sample level, to enlarge the diversity of training samples, several principles are tailored to revise existing training samples without actually implementing training sample collection. At the model level, two strategies are performed to add uncertainty to model parameters and encourage the model parameters to be more robust (Section IV).

Lastly, we conduct extensive experiments on various benchmarks under different workloads and data shifts. When workloads exhibit shift on query frequencies and query predicates, `GenIA` consistently surpasses the near-optimal method Extend [4] by about 12.7% while requiring less than 1% of the inference runtime. Compared with SOTA learning-based IAs, `GenIA` always demonstrates stronger robustness in different shift settings. Specially, When 63% of testing workloads are unseen, `GenIA` surpasses SWIRL [10] by 25%. When data dramatically changes, `GenIA` also surpasses SWIRL by 30%. The average training time required for `GenIA` is merely 37.1% of that needed for SWIRL.

II. Index Advisor Model

A. Generative Architecture

Most existing learning-based IAs are based on the Reinforcement Learning (RL) paradigm, which faces the convergence problem because it is tricky to balance exploration (i.e., trying new indexes) and exploitation (i.e., using known good indexes to learn the indexing policy). Too much exploration prevents convergence, and too much exploitation leads to sub-optimal indexes. Thus, most RL-based IAs can not guarantee a good result.

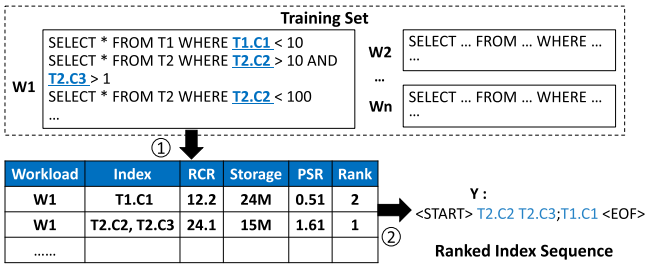

Fig. 2. Illustration of ranked indexes.

Since real-production database systems have already accumulated index creation samples on historical workloads [13], we can consider building IAs based on the Supervised Learning (SL) paradigm. The advantage of SL is that IA can instructively and steadily learn index selection. Once trained, the IA can give direct index recommendations with a small inference overhead.

The SL paradigm can be conducted as *discriminative*, i.e., *learning to classify* if an index is optimal for a given workload and dataset, or as *generative*, i.e., *learning to generate* the index. A discriminative model needs positive and negative samples. Unfortunately, due to the NP-hard nature of the index selection problem, the optimal indexes (i.e., positive samples) are too expensive to obtain. The training data will be highly imbalanced with non-distinctive positive and negative samples, which will coarsen the discrimination boundary. Furthermore, the existence of multi-column indexes makes a large candidate space, causing difficulty in learning a complex discrimination boundary. Thus, we are motivated to design a *generative model* without relying on classifying positive and negative samples.

B. Training Sample

As shown in Fig. 3, each training sample of GenIA is a pair of input and output, i.e., $< \mathbf{X}, \mathbf{y} >$. $\mathbf{X} \in \mathcal{R}^{\mathrm{n}_{column} \times \mathrm{n}_{feature}}$ is a feature matrix, where n_{column} is the number of indexable columns and $\mathrm{n}_{feature}$ is the feature dimension of each indexable column. $\mathbf{y} =< \mathbf{y}_1, \mathbf{y}_2, .., \mathbf{y}_r >$ is a sequence of arbitrary length. Each training sample is transformed from a record in the historical index creation experience, i.e., $< w, d, \mathcal{I}^{w,d} >$, where w is the workload, d is the data, and $\mathcal{I}^{w,d} = \{I_1, \ldots, I_n\}$ is the created indexes in the record. The input $\mathbf{X}$ is transformed from w, d by the feature extraction module (more details in Section III) to represent the workload and data characteristics.

Transformation of y. For each record, we first sort the indexes $I_i \in \mathcal{I}^{w,d}$. Sorting the indexes has never been implemented in the stage of training sample construction in previous IAs, but we argue this is a necessary step. Because under a limited storage budget, index selection faces the trade-off between storage and performance, i.e., a good index may be inferior to multiple sub-optimal small indexes due to excessive storage consumption. Thus, we consider the performance/size ratio [4], [5], [14] of each index I_i, which is computed as $PSR(I_i) = RCR(I_i)/storage(I_i)$, where $RCR(I_i)$ is the Relative Cost Reduction [10] (i.e., $1 - \frac{cost_{w/\ I_i}}{cost_{w/o\ I_i}}$) brought by index I_i, and $storage(I_i)$ is the storage cost of index I_i. The purpose of sorting is to give the IA more clues to the effectiveness of indexes, enabling the IA to generate indexes from good to bad without exceeding the storage budget.

Next, we decompose each multi-column index into several indexable columns. We use the token ; to segment different indexes and use a space to segment the indexable columns in a multi-column index. Thus, each output $\mathbf{y}$ is a sequence of indexable columns, starting from token <START> and ending by token <EOF>. This means that all training samples share the same vocabulary, i.e., including all indexable columns and special tokens, and the IA can make direct index recommendations for all workloads and data. Furthermore, the universal vocabulary naturally allows GenIA to capture the relationship between single-column indexes and multi-column indexes. This understanding can help the model to generate the most suitable index flexibly. On the contrary, RL-based methods [7], [8], [9], [10] treat single-column index and multi-column index by two state matrices with inevitable semantic gaps, which will decline the accuracy in predicting the next action.

As shown in Fig. 2, for the index configuration $\mathcal{I} = \{I_1 = [T_1.c_1], I_2 = [T_2.c_2, T_2.c_3]\}$, if $PSR([T_2.c_2, T_2.c_3]) > PSR([T_1.c_1])$, then we have $y =$ "< START> $T_2.c_2\ T_2.c_3$; $T_1.c_1$ < EOF>".

C. Encoder

Inspired by the Transformer model [15], which has been proven successful in many natural language generation tasks, GenIA is designed as an auto-regressive encoder-decoder network, as shown in Fig. 3.

One of the key mechanics in Transformer is the *scalable dot-product attention*, which computes a query matrix $\mathbf{Q}$, a key matrix $\mathbf{K}$ and a value matrix $\mathbf{V}$ respectively, for the input matrix $\mathbf{X}$ by $\mathbf{Q} = \mathbf{W^Q}X$, $\mathbf{K} = \mathbf{W^K}X$, and $\mathbf{V} = \mathbf{W^V}\mathbf{X}$, where $\mathbf{W^Q}, \mathbf{W^K}, \mathbf{W^V}$ are learnable parameters. The attention score $Attn(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ computes a weighted sum of the value matrix $\mathbf{V}$, where the weights are computed by the attention scores, i.e., $\mathbf{Q}\mathbf{K}^T/\sqrt{d_k}$.

However, the attention in Transformer spans all indexable colu- mns. Certain columns that are unlikely index candidates will dilute the effectiveness of attention mechanisms. Existing studies [16], [17], [18] have shown that sparse attention and segmented attention reduces the "distraction" of irrelevant features by selectively focusing on important features.

Furthermore, we should differentiate column relations based on the tables. For example, a multi-column index typically operates on different columns within a single table. Since our goal is to generate multi-column indexes and single-column indexes from the universal dictionary of indexable columns, it is essential to separate the attention mechanisms for columns within the same table and columns across different tables, as separate attentions help the model to divide the indexes more effectively.

Therefore, as shown in the left of Fig. 3, we propose an Intra-Inter-Attention (**IIA**) strategy. Specifically, we define intra-table attention as the relationship between the indexable columns of

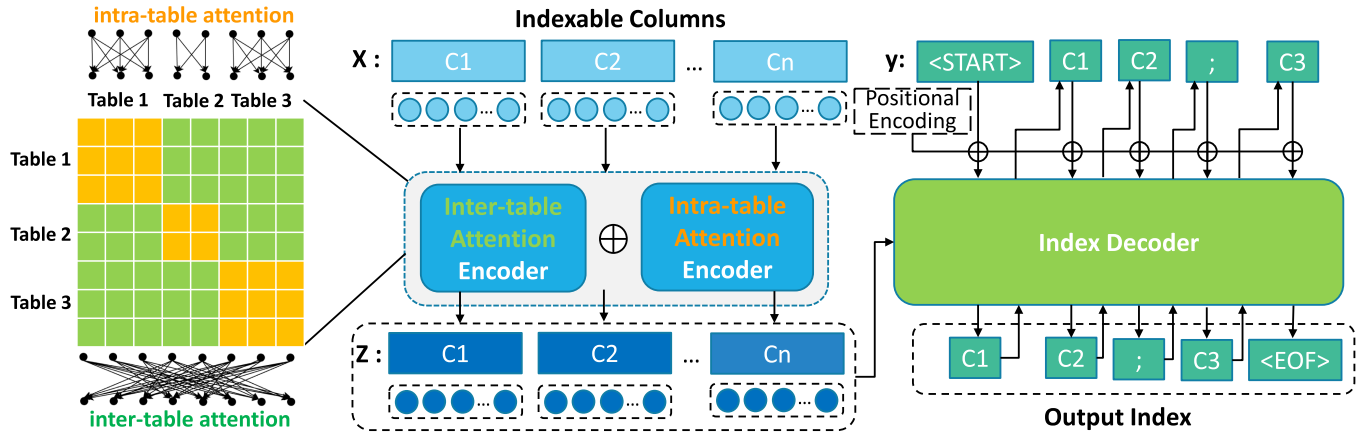

Fig. 3. GenIA Structure. The encoder is based on two Transformer encoders with inter-table and intra-table attention, respectively. The index decoder is a Transformer decoder.

the same table. Inter-table attention focuses on the interaction between the indexable columns across tables.

In computing the intra-table and inter-table attention, we build two mask matrices $\mathbf{M}_{intra}$ and $\mathbf{M}_{inter}$ and mask out (setting values to $-\infty$) all unwanted relations in the input of the Softmax.

$$Attn_o(\mathbf{Q},\mathbf{K},\mathbf{V},\mathbf{M}_o) = softmax\left(\frac{\mathbf{Q}\cdot\mathbf{K}^T}{\sqrt{d_k}}\odot\mathbf{M}_o\right)\mathbf{V}$$

$$,o\in\{\text{intra, inter}\},$$

$$\mathbf{M}_{\text{intra}}(i,j) = \begin{cases} 1, & \text{if i and j in the same table} \\ 0, & \text{otherwise} \end{cases}$$

$$\mathbf{M}_{\text{inter}}(i,j) = 1 - \mathbf{M}_{\text{inter}}(i,j). \quad (1)$$

Consequently, we have two encoders: an inter-table attention encoder that uses $Attn_{inter}$ and an intra-table attention encoder that uses $Attn_{intra}$. Each encoder consists of N_{enc} ($N_{enc} = 12$) layers similar to Transformer, except with a different attention computation. Unlike traditional Transformer, we do not add positional encoding because the input is a feature matrix instead of a sequence.

Finally, for the output of the inter-table attention encoder $\mathbf{Z}_{inter}$, and the intra-table attention encoder $\mathbf{Z}_{intra}$, we add them as the output $\mathbf{Z}$ of the whole encoder part:

$$\mathbf{Z} = \mathbf{Z}_{inter} + \mathbf{Z}_{intra}. \quad (2)$$

D. Index Decoder

The decoder also follows the Transformer architecture, with N_{dec} ($N_{dec} = 12$) layers. As shown in Fig. 3, the vocabulary is embedded into vector representations. For each sample, the decoder starts with the initializing token <START>. In generating the next token, the decoder adds the positional encoding of each token in the previously generated sub-sequence to the masked multi-head attention layer. Another input is the output of the encoder Z, which is fed to the multi-head attention layer. The final output is processed in the softmax layer to generate a possibility distribution over the vocabulary. Finally, we adopt a greedy decoding strategy that selects the index (token) with the highest probability at each step, which cannot guarantee the generation of the optimal index configuration. The decoding process terminates once the budget is exceeded, thereby ensuring flexibility within budget constraints.

III. Representing Dynamic Workload and Data

For GenIA to recognize and adapt to dynamic database environments, we extract workload and data features, which are significantly different from all previous IAs that only incorporate workload features [7], [8], [9], [10]. The feature extraction module derives two feature matrices, i.e., $\mathbf{M}^{workload} \in \mathcal{R}^{\mathrm{n}_{column}\times\mathrm{n}_{workload}}$ for workload features and $\mathbf{M}^{data} \in \mathcal{R}^{\mathrm{n}_{column}\times\mathrm{n}_{data}}$ for data features. The two feature matrices have the same number of rows, i.e., n_{column}, which is the number of indexable columns. The advantage is that we can directly concatenate $\mathbf{M}^{workload}$ and $\mathbf{M}^{data}$. Thus, it allows us to integrate the workload and data features seamlessly and capture the feature interactions. Moreover, the combination is still a vectorized representation of indexable columns. Learning indexable column generation from representations of indexable columns is more natural.

A. Workload Featurization

Given a workload $w = \{(q, f_q)\}$, with a set of queries, each query q has a frequency f_q. The query can be converted into a query plan, which is important in describing the query features [11]. Thus, the workload feature matrix consists of two parts: the first part is extracted from the query plan to identify potential indexes for the query operations, and the second part contains other features not extracted from the query plan. The first part (i.e., *plan features*) contains n_{plan} features and the second part (i.e., *auxiliary features*) contains $\mathrm{n}_{auxiliary}$ features, i.e., $\mathrm{n}_{workload} = \mathrm{n}_{plan} + \mathrm{n}_{auxiliary}$.

Plan Features. Since a query plan tree contains multiple nodes, each of which has a *Node Type* (e.g., "Merge Join" or "Seq Scan"), several key attributes, including *estimated statistics* (e.g., "Cardinality"), and *filter conditions* (e.g., "Merge Cond" or "Filter") on various filter predicates. We can extract critical

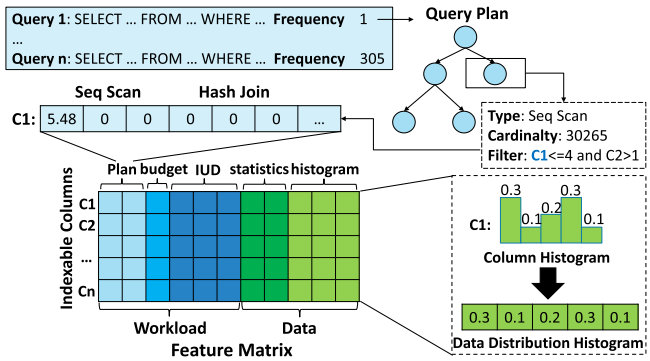

Fig. 4. Feature extraction of workload and data.

cost information from the attributes of each node to represent the characteristics of indexable columns. Specifically, for each node type, the filter condition affects the selection of indexes. When multiple filter predicates are applied, multi-column indexes may be beneficial, and the order of the predicate matters. Therefore, we set the dimension of the plan features to be $\mathrm{n}_{plan} = \mathrm{n}_{node} \times \mathrm{n}_{max}$, where n_{node} is the number of node types and n_{max} is the maximal index length.

To obtain the plan features, we traverse all queries. For each query with frequency f_q, we convert it into a query plan under an empty index configuration. Then, we traverse all nodes in the query plans, obtain the node types and cardinalities, and fetch filter predicates with their positions. We consider at most n_{max} filter predicates at each node's filter condition. We aggregate key information over all nodes of the same node type with the same indexable column at the same position.

$$\mathbf{M}^{workload}_{j,(t(i)-1)\times n_{max}+k(j,\Gamma(i))} = \sum_{i\in q, q\in w} log\left(f_q \times c(i)\right), \quad (3)$$

where q is a query in the workload $q \in w$, i is a node in q's query plan $i \in q$ with type $t(i)$, filter condition $\Gamma(i)$ and cardinality $c(i)$. j is an indexable column, and it appears in the filter condition $\Gamma(i)$ at position $k(j, \Gamma(i)) \leq \mathrm{n}_{max}$.

For example, as shown in Fig. 4, for the "Seq Scan" node of the current query plan tree with frequency 1, the first filter predicate is "C1 <= 4". The cardinality is 30265. We calculate $log(30265 \times 1)$ and get 5.48. The corresponding entry, i.e., the first element of "Seq Scan" in the row "C1", is added by 5.48.

Auxiliary Features. To represent the constraint condition, We assign one component for the indexing budget in every row of $\mathbf{M}^{workload}$. The budget value is the same for each indexable column. Data manipulation statements such as Insert, Update, and Delete (IUD statements) can impact the index selection, e.g., cause additional index maintenance costs. To more accurately capture the impact of transactional queries on index performance, we assign one component for each of the three IUD statements. Therefore, the dimension of the auxiliary features is $\mathrm{n}_{auxiliary} = 4$.

To obtain the auxiliary features, suppose the storage budget is b, we assign $\mathbf{M}^{workload}_{j,\mathrm{n}_{node}\times\mathrm{n}_{max}+1} = log(b)$, where j is an indexable column. We append each IUD statement's cardinality and frequency information to its operated columns.

$$\mathbf{M}^{workload}_{j,\mathrm{n}_{node}\times\mathrm{n}_{max}+1+t(s)} = \sum_{j\in s, s\in w} log\left(f_s \times c(s)\right), \quad (4)$$

where s is an IUD statement in the workload $s \in w$ with frequency f_s. $c(s)$ is the cardinality of the root node of the query plan s. j is an indexable column, and s operates on column j, i.e., $j \in s$. $t(s)$ is the type of s. $t(s) = 1, 2, 3$ for the Insert, Update, and Delete statements, respectively.

B. Data Featurization

The data feature matrix $\mathbf{M}^{data}$ compresses key information regarding the data d. The data feature matrix also contains two parts. Firstly, the data features should be able to roughly describe the state of the database that directly influences the indexing selection. In addition, to cope with dynamic data in Hybrid Transactional and Analytical Processing scenarios, the data distribution should be encoded to reflect the data shifts. Thus, $\mathrm{n}_{data} = \mathrm{n}_{influence} + \mathrm{n}_{distribution}$, where $\mathrm{n}_{influence}$ is the dimension of the first part (i.e., *data influence feature*), and $\mathrm{n}_{distribution}$ is the dimension of the second part (i.e., *data distribution feature*).

Data Influence Feature. We assign three components (e.g. $\mathrm{n}_{influence} = 3$) and use statistical information including *index selectivity*, *the number of table records*, and *the estimated storage cost* for each indexable column. Specifically, for the column i with index selectivity sel_i, the number of table records rs_i and the estimated storage cost cs_i if a single-column index is built on i, we calculate: $\mathbf{M}^{data}_{i,1} = sel_i$, $\mathbf{M}^{data}_{i,2} = log(rs_i)$ and $\mathbf{M}^{data}_{i,3} = log(cs_i)$.

Data Distribution Feature. As shown in Fig. 4, we calculate an up-to-date data histogram for each indexable column because the data histogram can effectively reflect the data distribution. Suppose we divide the values of column i into $\mathrm{n}_{distribution}$ ($\mathrm{n}_{distribution} = 10$ in default) bins of equal width, $\mathbf{M}^{data}_{i,3+j} = log(rs^j_i)$, where rs^j_i is the number of records in the $j-th$ bin. A larger number of bins $\mathrm{n}_{distribution}$ can capture subtle data shifts. However, it will cost more construction time and computational resources, and it will overcomplicate the data and the learning process of `GenIA`.

IV. Perturbation-Based Training Strategy

Generalization refers to a learning-based IA's ability to accurately select indexes for workloads and data unseen in the training phase. The generalization of a machine learning model has always been a difficult problem. Although a few recent IAs [8], [9], [10] seek to generalize to unseen workloads, their performances are still far from satisfactory. The generalization of learning-based IAs to new data has never been explored. When an IA generalizes poorly to new workloads and data, two outcomes are expected: either the selected indexes are ineffective, or the IA needs to be re-trained or updated before making the index selection. In either case, it hurts the real-time performance of a database system.

A. Perturbation-Based Training Sample Augmentation

To improve GenIA's generalization ability, our first effort is to perform *training sample augmentation* to enlarge the diversity of the training sample. Training sample augmentation is commonly adopted [19], [20] to improve generalization ability because it exposes the model to more samples and prevents it from over-fitting the noise in training data. However, we focus on the index selection problem, which differs significantly from other domains. Novel perturbation principles are demanded to reflect indexing properties and ensure the workloads are executable. Moreover, it is important that our perturbation does not incur additional data collection costs.

Our training samples originate from the historical index creation records, where each record contains a workload w, a dataset d, and the corresponding index configuration $\mathcal{I}^{w,d}$. Naturally, we can perturb a random training sample's *workload* and *data* and assign reasonable index configuration.

1) Workload Perturbation: Since each workload consists of a set of queries and their frequencies, workload perturbation can be implemented on queries and frequencies. Note that, unlike the query rewriting problem, we perturb the workload to make GenIA see more workload patterns without maintaining the semantic consistency of the workload before and after perturbing.

Frequency Perturbation. Workloads in real-world scenarios often exhibit fluctuations in frequency over a short period. To simulate workload bursts, we use two ways to alter the frequencies associated with the original workload.

(1) We randomly pick a workload and a scale factor ($\zeta \in (0, 10)$) to multiply the frequency of all queries in the workload. This step will simultaneously scale up or down the frequencies of all queries. Since the relative ratio of frequencies between each query remains constant, we can keep the original index configuration.

(2) There exist workloads with the same queries but different frequencies. We call them monozygotic workloads. We randomly pick a pair of monozygotic workloads w^1 with index configuration $\mathcal{I}^1$, w^2 with index configuration $\mathcal{I}^2$. Then, we randomly generate a coefficient $r \in (0-1)$ and generate a new workload $w' = \{(q', f_{q'})\}$, where the query set is the same as w^1, w^2, the query frequency is $f_{q'} = r \times f_q^1 + (1-r) \times f_q^2$, f_q^1 is the query frequency in w^1 and f_q^2 is the query frequency in w^2. To determine the index configuration for w', we estimate the performance of $\mathcal{I}^1, \mathcal{I}^2$ to w' by calling the what-if optimizer and decide the better index to be the new workload's index, e.g., $\mathcal{I}' = \mathcal{I}^1$, if $\mathcal{I}^1$ is better than $\mathcal{I}^2$, otherwise, $\mathcal{I}' = \mathcal{I}^2$.

Query Perturbation. In addition to frequency perturbation, we consider modifying the queries to add diverse query patterns. Conditional predicates are often the most influential factor in index selection. Therefore, we can generate new workloads by perturbing the predicates.

(1) We perform column swaps to generate a new workload with a different column order. Specifically, we identify two indexable columns i, j of the same attribute type (e.g., integer) from the index configuration. We then verify whether these columns can be exchanged by calling the what-if function. For workload w with index configuration $\mathcal{I}$, if workload after the column swap is executable, we swap i, j in each query of w and the index configuration $\mathcal{I}$. In this manner, we obtain a new workload w' and a new index configuration $\hat{\mathcal{I}}$. Then, we estimate the performance of $\mathcal{I}, \hat{\mathcal{I}}$ for w' by calling the what-if optimizer. The index configuration $\mathcal{I}'$ is defined as the better of $\mathcal{I}, \hat{\mathcal{I}}$.

(2) To explore query patterns unrevealed in the current training set, we can replace the conditional predicates with an unused column. Specifically, we randomly select a column i from the index configuration. Then, we select another column j which is (1) of the same type as i (e.g., integer); (2) not used as index configurations in the training set; (3) with more records, i.e., $rs_i \leq rs_j$, where rs is the number of records, which indicates that j is from the same or a larger data table; (4) with index selectivity $sel_j \geq 0.5$. We replace i with j in every query of the workload and index configuration.

2) Data Perturbation: The underlying data of a database in real-world scenarios is often constantly changing. To enrich the diversity of training data regarding data volume, we can refine the components in the data feature matrix $\mathbf{M}^{data}$. These components include the number of table records, the estimated storage cost, and all elements in the data distribution features. Specifically, we randomly choose a scaling factor, which is a positive integer that is less than 10, and multiply $\mathbf{M}^{data}_{i,2}, \mathbf{M}^{data}_{i,3}, \mathbf{M}^{data}_{i,3+j}$ for all columns i and data histogram bins $j \leq \mathrm{n}_{distribution}$. Since the scaling does not change the data distribution, we keep the index configuration unchanged.

The above perturbation methods are illustrated in Fig. 5. The workload perturbation is implemented on the original workload and transformed into workload features. The data perturbation is implemented directly on the data feature matrix. We add a percentage ω of perturbation data evenly using the above perturbation methods.

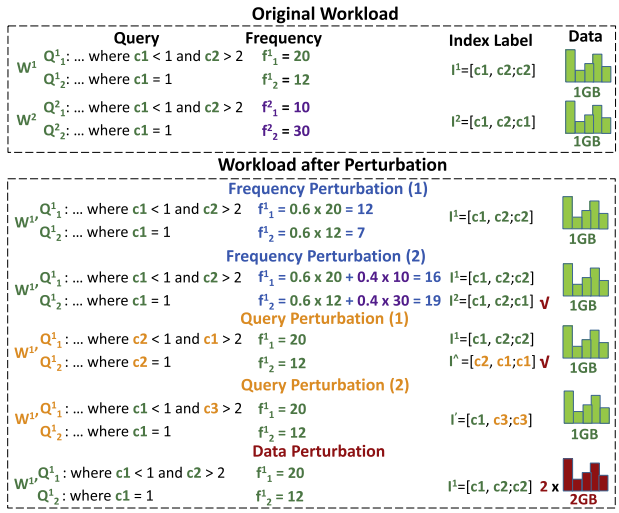

Fig. 5. Illustration of perturbations at the training data.

B. Perturbation-Based Model Augmentation

Including all workload and data patterns in the training set is impossible. Therefore, only perturbing the training data is

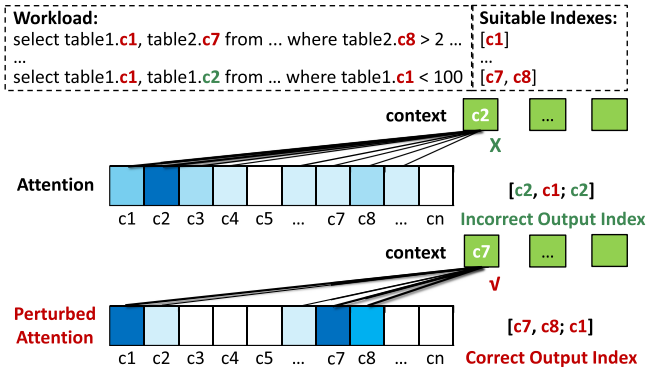

Fig. 6. Illustration of attention perturbation.

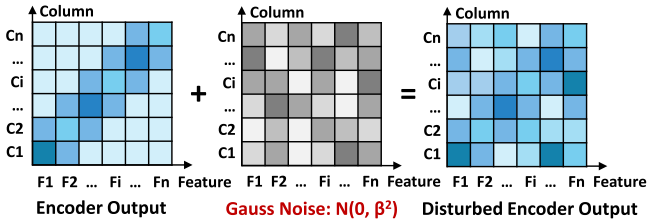

Fig. 7. Illustration of perturbations at encoder output.

insufficient. We also perform perturbation on the model parameters. The idea of model perturbation is to introduce uncertainty to the model without any extra training to force the model to be more robust to different input.

There are two important types of model parameters in GenIA: one is the calculation of attention in both the encoder and decoder; the other is the encoder output. The former encapsulates the implicit relationship between workload/data and index. The latter encodes concrete input information and directly affects the decoder's generation. Thus, we design two perturbation strategies.

1) Attention Perturbation: We consider adding perturbations to the attention computation of GenIA to improve GenIA's performance and generalization ability. Our perturbation method is to sparsify the attention score matrix randomly. Specifically, at each layer of the encoder and decoder during the forward propagation of each training epoch, we randomly generate a matrix $\mathbf{M}_{random}$. Then, we randomly sample a binary matrix $\mathbf{M}_{pert}$ with the same dimension as the attention score matrix.

$$\mathbf{M}_{pert}(i,j) = \begin{cases} 0, & \text{if } \mathbf{M}_{random}(i,j) < \alpha \\ 1, & \text{otherwise} \end{cases} \tag{5}$$

where α is a hyper-parameter. We use $\mathbf{M}_{pert}$ to sparsify the attention matrix. The output of each attention layer is

$$Attn(\mathbf{Q},\mathbf{K},\mathbf{V},\mathbf{M}_{pert}) = softmax\left(\frac{\mathbf{Q}\cdot\mathbf{K}^T}{\sqrt{d_k}} \odot \mathbf{M}_{pert}\right)\mathbf{V} \tag{6}$$

As shown in Fig. 6, sparsifying the attention scores encourages the model to focus on relevant tokens and generate correct indexes. Randomly removing certain attention scores encourages the model to learn more diverse and adaptable patterns, preventing it from becoming overly reliant on specific parameters that would cause overfitting on training data.

2) Encoder Output Perturbation: The input of a training sample is finally transformed into the output of GenIA's encoder $\mathbf{Z}$. We can also directly manipulate Z since it is essentially a high-level abstraction of a training sample, and directly operating on it can be regarded as a means of data enhancement.

Therefore, we consider adding a small enough Gaussian noise to $\mathbf{Z}$ to simulate the variation of training data at the feature level. Random noise can add uncertainty to the input and force the model to be independent of some specific encoding features in the training, thereby improving the model's generalization ability to unseen features.

Specifically, in the forward propagation of each training epoch, we add a trivial Gaussian noise to every output $\mathbf{Z}$:

$$\mathbf{Z}' = \mathbf{Z} + \epsilon,\ \epsilon \sim \mathcal{N}(0, \beta^2), \tag{7}$$

where ϵ has the same dimension as $\mathbf{Z}$, and the hyperparameter β is the standard deviation of the Gaussian noise. As shown in Fig. 7, the random noise ϵ disrupts the encoder output.

V. Experiment

We conduct extensive experiments to evaluate GenIA and answer the following questions:

- **Q1**: Is GenIA efficient and effective in AP (Analytical Processing) and Hybrid Transactional and Analytical Processing scenarios, with and without workload and data shifts? (Section V-C)
- **Q2**: How does GenIA's performance vary under different workload and data shifts, concerning
  - **Q2.1**: the proportion of unseen templates in the testing workloads (Section V-D),
  - **Q2.2**: the ratio of read/write transactions in Hybrid Transactional and Analytical Processing workloads (Section V-E),
  - **Q2.3**: the volume divergence between training data and testing data (Section V-F),
  - **Q2.4**: gradual workload shifts (Section V-G).
  - **Q2.5**: real-world workload shifts (Section V-H).
- **Q3:** How does each component in GenIA contribute to its performance? Specifically,
  - **Q3.1**: Does the novel model design of GenIA, e.g., inter-column and intra-column attention, perturbation-based training process benefit index selection? (Section V-I)
  - **Q3.2**: Does the feature extraction module help GenIA generate better indexes? (Section V-J)
  - **Q3.3**: How do GenIA's model parameters affect its performance? (Section V-K)
  - **Q3.4**: How do index labels, including index ranking methods and the quality of index labels affect GenIA's performance? (Section V-L)

A. Experiment Setup

Benchmark. We experiment with AP and Hybrid Transactional and Analytical Processing scenarios. For the *AP scenario*, we use three benchmarks: (1) TPC-H [21], which contains 8 tables and 61 columns; (2) TPC-DS [22], which contains 25 tables and 429 columns; (3) JOB [23], which contains 21 tables and 108 columns. Among these datasets, TPC-DS and JOB feature large schemas and high-dimensional workloads, which pose greater challenges for index recommendation. For the *Hybrid Transactional and Analytical Processing scenario*, we use CH-BenCHmark [3], [24], which combines the TPC-C [25] schema and three tables from TPC-H. It comprises TPC-C transactional workloads and 22 queries modified from TPC-H queries to adapt to the CH-BenCHmark schema.

Workloads. The workloads, either for training or testing, are composed of queries generated from query templates and frequencies (more details in Section V-B1). We follow [10] to generate workloads. Specifically, we exclude some benchmark query templates that are too expensive to execute. Otherwise, the index selection problem will be oversimplified.

In line with [10], we exclude TPC-H query templates $2, 17, 20$ (the same for the CH-BenCHmark) as well as TPC-DS query templates $4, 6, 9, 10, 11, 32, 35, 41, 95$. For each workload, we randomly choose $|w|$ templates from the remaining available benchmark templates and populate queries. $|w| = 5$ for TPC-H. $|w| = 50$ for TPC-DS. $|w| = 30$ for JOB. For the CH-BenCHmark, we select five AP templates and all TP (Transaction Processing) templates. We assign each query a frequency randomly drawn from [1,10000].

Data. Unless otherwise stated, we generate 1 GB of data in each benchmark to train and evaluate IAs (3.6 GB for JOB). We also consider evaluating with dynamic data, where the data volumes and distributions differ in training and testing (more details in Section V-B2).

Evaluation Metrics. For *AP scenarios*, we measure each IA by *Relative Cost Reduction (RCR)*, which computes the ratio of reduced workload cost. For *Hybrid Transactional and Analytical Processing scenarios*, we adopt two metrics. (1) *Latency*, which is the actual query runtime with the selected indexes; (2) *Throughput*, which is the number of queries or transactions processed.

Competitors. We compare `GenIA` with ten well-known IAs, including two *heuristic-based* IAs and eight *learning-based* IAs. Specifically, the `heuristic-based` IAs include: (1) Extend [4], which iteratively updates the index configuration from the initial empty index configuration and eventually produces a near-optimal index configuration; (2) DB2Advis [5], which considers index candidates by decreasing benefit-per-space ratio, and performs random variations. The *learning-based* IAs include: (3) DQN [7] and (4) DRLindex [8], [9] which use the Deep Q-Network algorithm to recommend indexes by conducting multiple indexing trails on each workload; (5) SWIRL [10], which uses a proximal policy optimization (PPO) algorithm; (6) DBA bandits [1], which models the index selection problem as the multi-arm slot machine problem; (7) AutoIndex [6], which builds a Monte Carlo tree to find the optimal index; (8) BALANCE [26], which conducts transfer learning from multiple small-scale historical versions; (9) λ-Tune [27], which leverages LLMs for database tuning; and (10) MFIX [28]: which employs Bayesian optimization for data-efficient index exploration via both estimated and actual execution. We implement DRLindex based on the original paper [8], [9] and adopt open-source implementations for the rest. Note that both versions of the other supervised IA (i.e., IdxL [13]) can not be used as baselines because they are only applied to slow queries and infeasible for recommending workload indexes.

Training and Testing. (1) For *heuristic-based* IAs, no training is needed. (2) We train `GenIA` with 2,000 indexing records (before perturbation), each consisting of the workload, data, and the index configuration. We use Extend [4] for real execution to assign the ground-truth index configuration. Note that the Extend's output is not guaranteed to be optimal. However, `GenIA` is a generative model, and it does not depend on high-quality positive (i.e., truly optimal) and negative (i.e., truly bad) samples as classification methods. (3) To train other competitors, we use the same number of training workloads as `GenIA`. For each workload, the number of trials is defined according to the original papers.[1] (4) In testing, we randomly generate 100 workloads and use real execution cost. Each experiment (under one setting) is repeated three times, and the average performance of each IA is reported.

Other Implementation Details. The storage budget is randomly picked from a fixed range (250-2000 MB for TPC-H 1 GB, TPC-DS and CH-BenCHmark, 500-4000 MB for TPC-H 2 GB and TPC-DS 2 GB, 750-6000 MB for TPC-H 3 GB and TPC-DS 3 GB, 1250-10000 MB for TPC-H 5 GB and TPC-DS 5 GB, 2500-20000 MB for TPC-H 10 GB and TPC-DS 10 GB, 900-7200 MB for JOB). All benchmark-defined secondary indexes are removed. For AP benchmarks, we employ psycopg2 [29] to interact with PostgreSQL, and HypoPG [30] for the what-if optimizer for the training of competitors. For the Hybrid Transactional and Analytical Processing benchmark, we use real execution cost for the training of competitors (containing maintenance cost). Our codes are available online.[2] As in previous studies [1], [6], we use the default index type of PostgreSQL.

Environment. Database indexing experiments are conducted with Python 3.7, PostgreSQL 12.5 [31] on a workstation with an Intel(R) Xeon(R) CPU E5-2678 v3 @ 2.50 GHz, 256 GB main memory and a GeForce RTX 2080 Ti graphics card. The machine learning models are implemented in a GPU server with an Intel Xeon Gold 6133@2.50 GHz CPU and a GeForce RTX 3090 Ti card.

B. Dynamic Setting

We can create a dynamic environment from both the workload and data perspectives. We define three shift levels on each perspective, leading to nine combinations.

[1] Notably, we do not give an unfair advantage to `GenIA` because the number of training samples is much smaller for `GenIA`.

[2] https://github.com/XMUDM/GenIA

1) Dynamic Workload: (1) **WL1: Workload Shift on Predicate and Frequency**. The training workloads and the testing workloads are generated from the same set of query templates, but the queries have different predicates, and each query has a different frequency. This level of workload shift reflects the most basic scenario. For example, customers of e-commerce websites may filter products based on different filtering values, and the frequency of queries fluctuates over time.

(2) **WL2: Workload Shift on Benchmark Template**. Following [10], **WL2** goes beyond **WL1** by letting the testing workloads contain γ ($\gamma = 30\%$ unless otherwise stated) of unseen benchmark-defined query templates that do not occur in the training stage. This shift level simulates the migration of information needs, e.g., users of e-commerce platforms search for different product aspects, including prices, brands, availability, and so on.

(3) **WL3: Workload Shift on Out-Of-Benchmark Template**. Following [12], the unknown templates are not benchmark-defined query templates. Instead, they are generated by FSM [32]. This level of workload shift simulates a burst event in the real world, e.g., customers search for deals on an unexpected promotion day, which is different from usual query patterns related to product specifics.

2) Dyanamic Data Setting: (1) **DL1: No Data Shift**. In AP scenarios, including the TPC-H, TPC-DS, and JOB benchmarks, we use the same data (i.e., TPC-H and TPC-DS is 1 GB, JOB is 3.6 GB) for training and testing. This is the default experimental setting in existing studies. Under **DL1**, the training and testing of the IAs depend on the workload shift levels and the IA architecture. Specifically, only DRLindex, SWIRL, BALANCE, and `GenIA` can directly generalize to unseen query templates. Thus, they can be trained using training workloads and make direct index recommendations on testing workloads. Other IAs lack the inherent mechanism to handle workload shifts quickly. Thus, for all workload shift levels, they need to be re-trained (or re-ran for the case of heuristic-based IAs) on the testing workloads.

(2) **DL2: AP Data Shift**. Following [11], this level of data shift simulates the scenario when the underlying data of the database changes. **DL2** is conducted on benchmarks TPC-H [21] and TPC-DS [22]. Note that JOB's data volume is fixed, so it is not involved in this level of data shift. First, we construct a pool of five datasets of data volume: 1 GB, 2 GB, 3 GB, 5 GB, and 10 GB. Then, we use the smallest dataset (1 GB) as the *target* and the remaining four datasets (2/3/5/10 GB) as the *source*. For `GenIA`, we collect 500 training samples from each source dataset. After `GenIA` finishes training, it is employed to directly predict indexes for the testing workload on the target dataset (i.e., 1 GB). For DRLindex, SWIRL, and BALANCE, they can only be trained on a fixed dataset, so we use 2000 samples on each source dataset to train four model copies. Then, we use the four model copies to recommend indexes on the target dataset. The best result is conserved as the final result. For other IAs, we train and test them on the target dataset. (see Section V-F for more detailed data shift experiments)

(3) **DL3: Hybrid Transactional and Analytical Processing Data Shift**. Following [1], this level simulates data shifts in Hybrid Transactional and Analytical Processing scenarios by running workloads of the CH-BenCHmark. Due to the large number of TP transactions being executed, DL3 will change the data distribution more significantly, which brings greater challenges to the IA's robustness in handling data shifts. Again, we construct a pool of five datasets by running the Hybrid Transactional and Analytical Processing workloads on the CH-BenCHmark. Specifically, the original dataset and the three resulting datasets by performing the workloads one to three times are the four source datasets. The target dataset is obtained by running the Hybrid Transactional and Analytical Processing workloads four times. `GenIA` is trained on the source and directly employed on the target dataset. DRLindex, SWIRL, and BALANCE are separately trained on four source datasets, and the best model copy is evaluated on the target dataset. Other IAs are trained and evaluated on the target dataset.

C. Indexing Efficiency and Effectiveness

Effectiveness. To answer **Q1**, we report the performance of various IAs under different workload and data shift levels. The experimental results are shown in Fig. 8. We observe:

(1) *On average,* `GenIA`*outperforms SOTA IA Extend across different settings by 7.5% on average, with less than 1% of the inference time.*

(2) In AP scenarios, with larger workload shifts or data shifts, all IAs without re-training experience performance degradation, which indicates that dynamic workloads and databases present a significant challenge to making direct inferences. However, `GenIA` improves performance by 19.2% and 16.5% over SWIRL and BALANCE, respectively. This shows that `GenIA` has a relatively strong ability to adapt to dynamic scenarios. Note that all heuristic-based IAs (i.e., Extend and DB2Advis) and some learning-based IAs (i.e., DQN, AutoIndex, Bandits, λ-Tune and MFIX) show stable performance because they undergo full retraining. Thus, their performance is obtained at the expense of large inference time, which will be discussed later. It is worth noting that the performance of MFIX is suboptimal. This is primarily because it is designed for static index recommendation scenarios, whereas dynamic scenarios introduce greater challenges due to the shifts of query frequencies, query templates, and underlying data distributions.

(3) In the scenario of Hybrid Transactional and Analytical Processing with transactional workloads, making direct index recommendations is particularly difficult. DRLindex, SWIRL and BALANCE exhibit low throughput and high latency. This is because they only consider AP workloads while ignoring the impact of transactional workloads on index selection strategies. Transactional workloads contain a large number of data manipulation statements, which not only introduce additional index maintenance overhead but also alter the underlying data of the database, posing greater challenges to the index selection problem. Existing index advisors which make direct index recommendations fail to effectively address these impacts. On the contrary, `GenIA` still achieves performance comparable to the optimal Extend algorithm. This is attributed to its ability to model the impact of data manipulation statements during

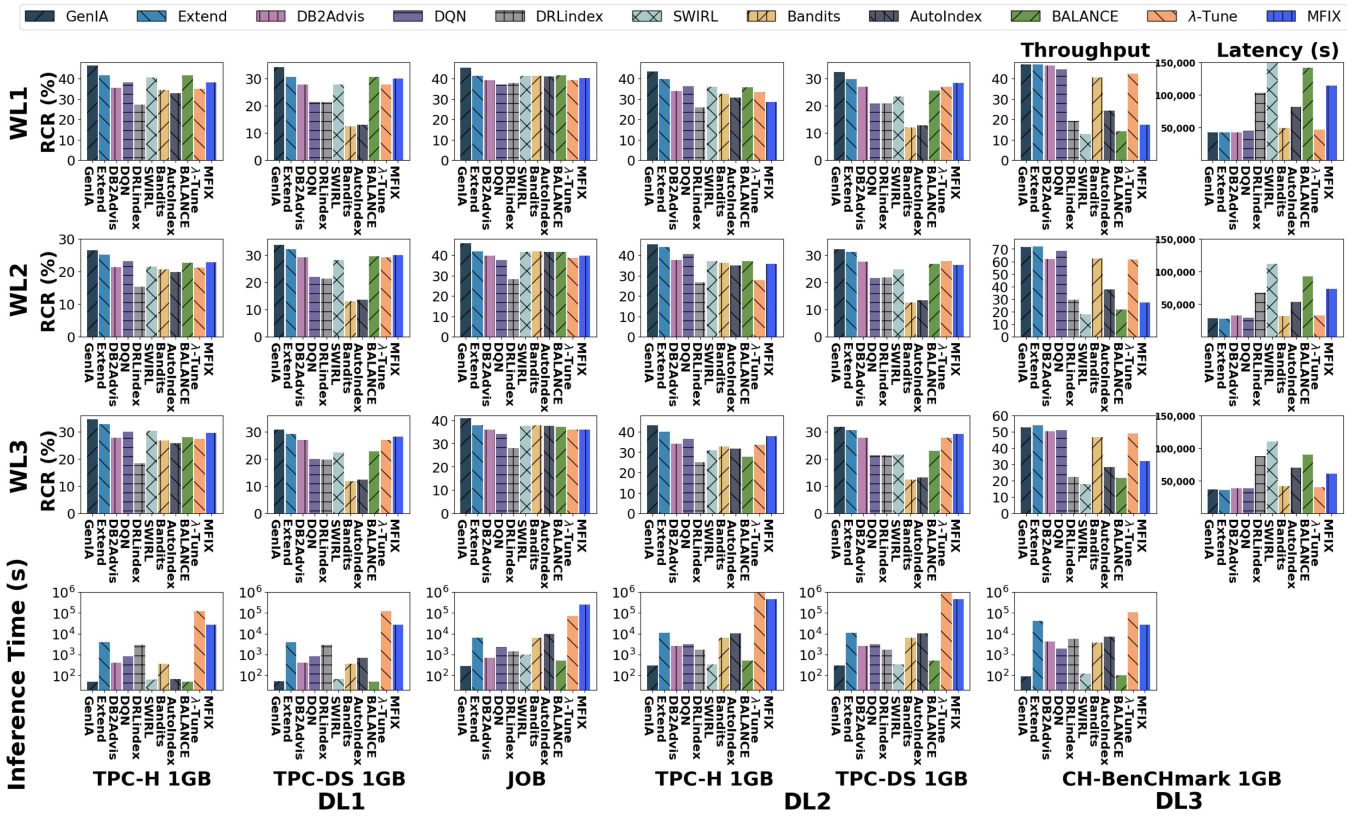

Fig. 8. Performance of IAs under workload and data shifts.

TABLE I
INFERENCE RUNTIME ON 100 WORKLOADS (SECONDS)

Dataset	TPC-H 1GB	TPC-H 10GB	TPC-H 100GB	TPC-DS 1GB	TPC-DS 10GB	TPC-DS 100GB
GenIA	**40**	**40**	**40**	**366**	**366**	**366**
SWIRL	64	67	72	540	667	803
DB2Advis	405	472	535	1405	2093	2767
Extend	2333	2685	3102	236923	278223	329045
DQN	842	875	1269	1975	2667	2810
DRLindex	3021	3885	4980	18967	27970	35529
AutoIndex	685	846	1069	10300	14181	19832
Bandits	904	1098	1332	7256	11239	18037
BALANCE	45	50	68	502	641	780
MFIX	92005	893233	>480h	1333702	1715046	>480h
λ-Tune	25131	382821	>480h	1656412	>480h	>480h

Note: h denotes hours.

TABLE II
TRAINING COST COMPARISON (HOURS)

Dataset	TPC-H 1GB	TPC-H 10GB	TPC-H 100GB	TPC-DS 1GB	TPC-DS 10GB	TPC-DS 100GB
GenIA	0.54	0.54	0.54	**2.41**	**2.50**	**2.69**
SWIRL	0.75	0.82	0.90	6.31	6.52	6.90
DQN	0.38	0.41	0.45	4.07	4.23	4.49
DRLindex	**0.26**	**0.28**	**0.31**	2.48	2.67	2.90
BALANCE	0.45	0.58	0.63	3.82	4.40	4.45

feature modeling, while leveraging data histograms to precisely capture variations in underlying data, indicating that GenIA possesses strong capability in handling Hybrid Transactional and Analytical Processing workloads.

Efficiency—Inference cost. We report the mean summed inference time over 100 workloads using different index advisors in Table I. Specifically, We randomly generate 100 workloads 3 times on the same database with different workload shift settings and calculate the average inference time. We observe that GenIA has the shortest inference time. Specifically, GenIA is three orders of magnitude faster than the near-optimal method Extend because the heuristic-based algorithm re-executes the expensive search algorithm each time. GenIA is up to $10 \sim 100\times$ faster than IAs that need to be re-trained because the re-training cost is included in the inference cost. In addition, GenIA is faster in making index selections than DRLindex, SWIRL and BALANCE, which also make direct inferences. GenIA costs 59.4% of the inference time of BALANCE, 55.1% of the inference time of SWIRL, and 1.3% of DRLindex. This is due to GenIA's simple and efficient network architecture and feature processing modules.

Efficiency—Training overheard. For all learning-based IAs that can make direct index recommendations, we further investigate their total training time in Table II. We observe that: (1) On TPC-H databases, the training time of DQN and DRLindex is relatively short because their model structure and state design are simple. However, their indexing performances are below average, as shown in Section V-C. GenIA's training time is generally shorter than SWIRL and BALANCE, but its performance is better than their. (2) GenIA achieves the shortest training time on the TPC-DS database over different data volumes, which shows the high efficiency of supervised learning.

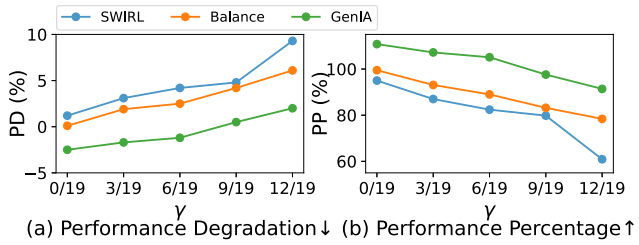

Fig. 9. Impact of proportion of unseen workloads.

TABLE III
PERFORMANCE W.R.T. DIFFERENT TRANSACTIONS

Method	Default		Write Heavy		Read Heavy	
	Throughput	Latency	Throughput	Latency	Throughput	Latency
GenIA	46.51	42902	65.07	30568	208.46	9499
Extend	46.88	42524	**69.39**	**28759**	**208.85**	**9388**
DB2Advis	**46.89**	**42372**	69.19	28842	208.43	9610
DQN	44.69	44617	33.04	60399	111.06	17655
DRLindex	19.21	103386	22.25	89699	40.26	49458
SWIRL	12.71	156259	11.79	168221	151.67	12927
Bandits	40.51	48817	64.90	30848	116.81	16788
AutoIndex	24.41	81362	44.51	44884	104.14	18830
BALANCE	14.06	141786	13.18	151363	156.64	12519
λ-Tune	45.94	46188	67.80	29431	200.34	9808
MFIX	17.42	114465	26.65	74893	167.77	11787

D. Performance w.r.t. Unseen Templates

In Section V-C, the proportion of unseen query templates in a testing workload is fixed to $\gamma = 30\%$. In this section, we further explore the impact of unknown templates. Experiments are conducted on the TPC-H under **WL2+DL1**. Following [10], we vary the number of unseen templates as 0,3,6,9,12 out of the total 19 templates.

To better quantify the performance recession, we use Extend as a baseline since it is not affected by unseen queries. We calculate two evaluation metrics as follows: (1) $PD_{IA} = RCR_{Extend} - RCR_{IA}$, where RCR is *Relative Cost Reduction*. This metric calculates the RCR degradation of the evaluated IA. Smaller PD suggests better stability. (2) $PP_{IA} = RCR_{IA}/RCR_{Extend}$. This metric calculates the performance of the evaluated IAs as a percentage of the performance of Extend. Higher PP suggests better performance.

The performance of SWIRL, BALANCE and GenIA is shown in Fig. 9. As the proportion of unseen templates increases, the performance of both IAs steadily declines. However, GenIA always surpasses Extend ($PP > 1$) until almost half ($\geq 9/19$) of the workloads are unseen. GenIA consistently outperforms SWIRL and BALANCE at all γ. Even with $12/19$ (i.e., 63.2%) of unseen query templates in a testing workload, GenIA still reduces PD by 65.6% and increases PP by 16.6% over BALANCE, showing that GenIA's superior ability to recommend indexes for dynamic workloads.

E. Performance w.r.t. Read/Write Transactions

The proportion of read/write transactions within an Hybrid Transactional and Analytical Processing workload significantly impacts the performance of IAs. To explore this, we conduct experiments on the CH-BenCHmark under **WL2+DL3** to examine how different read and write ratios affect IAs. We create three variations of testing workloads: (1) **Default**: Using the standard transaction proportions defined by the CH-BenCHmark. (2) **Write Heavy**: only contain transactions with high write rates, i.e., NewOrder and Payment. (3) **Read Heavy**: only contain transactions with high read rates, i.e., OrderStatus and Stock_Level.

The experimental results are shown in Table III. We observe that (1) GenIA's performance is comparable to the near-optimal Extend on default, write-heavy, and read-heavy workloads. Note that GenIA has maintained stable effectiveness while significantly improving the efficiency because GenIA has an inference time of $2-3$ orders of magnitude lower than Extend. (2) Heuristic-based IAs perform generally well on Hybrid Transactional and Analytical Processing workloads. (3) Except GenIA, the performance of other learning-based IAs is not stable. For example, SWIRL, BALANCE and MFIX produce better latency result on the read-heavy workloads while they performs poorly on write-heavy and default workloads. (4) The performance gap between heuristic-based IAs and learning-based IAs on the read-heavy workloads is smaller because the read-only workload is closest to the AP scenario, which most previous IAs are designed to serve. (5) In **Write Heavy** scenarios, the performance of GenIA is slightly inferior to that in the other two scenarios. This is because the massive transactional workloads in write-intensive scenarios incur substantial index maintenance costs and drastically alter the underlying data distribution of the database, making direct index recommendation extremely challenging. Other direct index recommendation methods, such as DRLindex, SWIRL, and BALANCE, suffer from severe performance degradation under such circumstances. In contrast, GenIA can still achieve suboptimal performance. In addition, although heuristic methods and the LLM-based method λ-Tune can also deliver competitive performance, this is attributed to the fact that they restart and execute the complete algorithm workflow from scratch rather than performing direct index recommendation, which incurs enormous inference overhead. The recommendation overhead of Extend and DB2Advis is more than 100 times that of GenIA, while the LLM-based method λ-Tune even requires over 1000 times the inference time of GenIA. Therefore, we argue that the slight performance degradation of GenIA is acceptable.

F. Performance w.r.t. Dataset Volume

To answer **Q2.2**, we construct variants of the source and target dataset by varying the source data volumes within the dataset pool under data shift level **DL2**. Specifically, we experiment with (1) **2/3/5/10 GB → 1GB**, (2) **2GB → 1GB**, (3) **1/2/3/5GB → 10GB**, and (4) **1/2/3/10GB → 5 GB**. Varying the data volume divergence between the source and target datasets allows us to investigate the impacts of data shifts on the IA's performance.

We report the performance of GenIA, SWIRL and BALANCE on TPC-H under all workload shift levels in Fig. 10. For both SWIRL and BALANCE, we train multiple model copies on

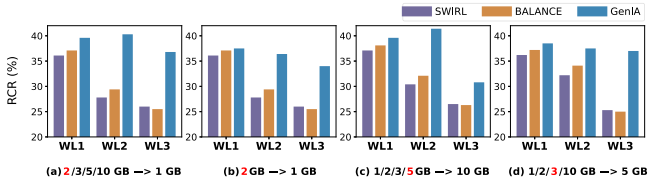

Fig. 10. Performance w.r.t. different dataset volume.

different source datasets and select the best-performing configuration. Interestingly, we observed that their best source dataset choices coincided in all cases. Therefore, we mark these overlapping best settings in red to emphasize the alignment. We have the following observations. (1) GenIA outperforms SWIRL and BALANCE in all data shift scenarios, especially at **WL2** and **WL3**, with an average improvement of more than 20%. This observation shows that GenIA transfers the knowledge gained on source datasets to the target dataset, thanks to the feature representation module that captures the data features. (2) When the target dataset is fixed, GenIA's performance increases with more diverse source datasets. For example, GenIA performs better on 2/3/5/10 GB $\rightarrow$ 1GB than 2GB $\rightarrow$ 1 GB. This observation also verifies that GenIA benefits from source datasets of varying volumes. (3) We find that SWIRL and BALANCE both have the best performance when they are trained on the source data volume that is closest to the target dataset. For example, if the target dataset is 1 GB, the best model copy is trained on 2 GB. This observation reveals that the data divergence limits their performance.

G. Performance w.r.t. Gradual Workload Shifts

To investigate GenIA's capability in handling gradual workload shifts, we follow [26] to synthesize a workload stream on TPC-H 1 GB. Specifically, we generate a workload stream consisting of 30 timestamps. In the first timestamp, we randomly select 5 templates to generate a workload with random predicate values and frequencies. In subsequent timestamps, we replace a random portion of the query templates from the previous workload. To ensure that learning-based IAs can undergo sufficient training, each timestamp contains 1,000 workloads. We then employ the Split-X% method, meaning that we sequentially merge adjacent timestamps into blocks until the difference between consecutive blocks exceeds X%. Learning-based IAs are trained on the previous block and tested on the next block, while heuristic-based IAs are executed directly on the test block.

Table IV reports the *Relative Cost Reduction (RCR)* of different IAs as the Split-X% threshold increases. We observe that: (1) The heuristic-based method, Extend, and learning-based IAs that require retraining (e.g., λ-Tune, MFIX) exhibit relatively stable performance. This is because their direct execution or retraining on the test workload significantly reduces the difficulty of index recommendation in gradual workload shifts. (2) However, GenIA still maintains the highest RCR as Split-X% increases from 0% to 40%. Even in the extreme case where Split-X% reaches 60%, GenIA achieves performance comparable to MFIX, demonstrating its strong ability to handle gradual workload shifts.

TABLE IV
RELATIVE COST REDUCTION UNDER DIFFERENT SPLITTING METHODS

Method	Split-0%	Split-20%	Split-40%	Split-60%
Extend	42.42	35.38	38.23	**40.51**
SWIRL	40.31	30.79	31.51	25.67
BALANCE	42.21	32.59	34.41	30.55
λ-Tune	36.05	30.22	32.57	35.42
MFIX	37.43	32.66	35.20	37.01
GenIA	**47.01**	**37.92**	**39.81**	37.23

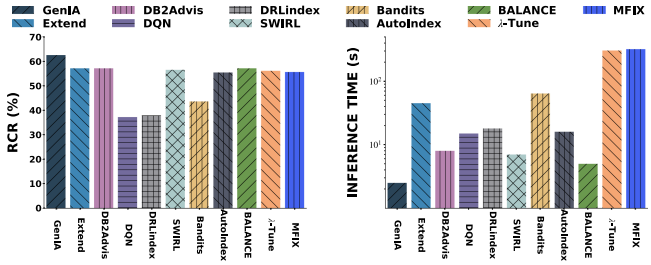

Fig. 11. Performance in real-world database.

H. Performance w.r.t. Real-World Workload Shifts

To investigate the performance of GenIA in real-world scenarios, we conducted experiments using the real-world dataset MATERIAL mentioned in [33]. MATERIAL is a large transactional database containing 79 tables and 1,265 columns, and provides real-world workload streams. We randomly sampled 1,000 workloads from the workload stream for GenIA's training, while keeping the training setups for other competitors unchanged. The storage budget for index selection was set to half the size of the database.

Fig. 11 presents the *Relative Cost Reduction (RCR)* and inference time of the IAs. We observe that: (1) Extend, DB2Advis, and most learning-based IAs (e.g., BALANCE, SWIRL, MFIX, etc.) exhibit similar and excellent performance, achieving an RCR of around 57%. This is because transactional workloads in MATERIAL are less complex than analytical workloads and do not include intricate IUD statements. (2) GenIA still demonstrates superiority by achieving over 11.2% better performance than the average of other methods with the shortest inference time on this real-world dataset.

I. Ablation on Model Architecture

In this section, we evaluate the impacts of the proposed novel model design, including the inter-column and intra-column attention mechanism and the perturbation-based training strategy. We use the *Relative Cost Reduction (RCR)* as the evaluation metric.

Firstly, for the impacts of the inter-column and intra-column attention mechanism, we conduct ablation experiments on the TPC-H benchmark under **WL1+DL1**, and design five different

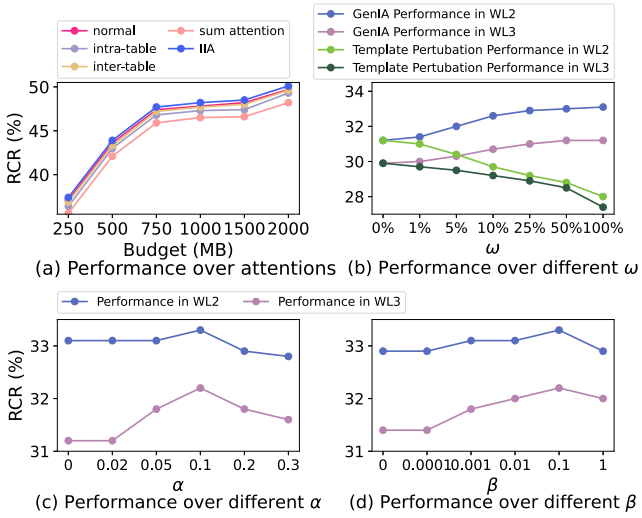

Fig. 12. Impact of model architecture.

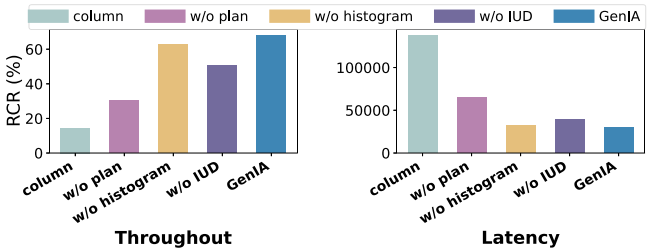

Fig. 13. Performance of different feature representations.

variants of attention mechanisms, including (1) **normal**, which is the vanilla attention mechanism in Transformer; (2) **intra-table attention**; (3) **inter-table attention**; (4) **sum attention**, which calculates the sum of two normal attentions, (5) **IIA, which is the full employment of attention mechanism in GenIA**.

The experimental results in Fig. 12(a) show that IIA achieves the highest performance over different storage budgets. The relative cost reduction by IIA is, on average, 7.1% higher than the vanilla global attention, verifying the effectiveness of integrating intra-table and inter-table column-wise relationships.

Secondly, for the impacts of the perturbation-based training strategy, we evaluate the impacts of various parameters in the perturbation-based training strategy, including the proportion of perturbation workload to the training set workload in the perturbation strategy (i.e., ω), the parameter of attention perturbation (i.e., α) and the standard deviation of Gaussian Noise (i.e., β) in encoder output perturbation. We measure the *Relative Cost Reduction (RCR)* on TPC-H benchmark under workload and data shifts **WL2+DL1** and **WL3+DL1**, and vary $\omega = 0\%$, 1%, 5%, 10%, 25%, 50% and 100%, vary $\alpha = 0$, 0.02, 0.05, 0.1, 0.2 and 0.3, and vary $\beta = 0$, 0.0001, 0.001, 0.01, 0.1 and 1. Note that setting these parameters to 0 corresponds to an ablation of that part. When discussing ω, we also investigate the impact of workload perturbation by adopting more diversified, template-level modifications. Specifically, we randomly select queries from each workload, rewrite them using the query generator IABART [12], and ensure that the indexable columns remain consistent before and after the transformation. Note that queries generated by IABART can yield higher diversity but cannot ensure a limited perturbation amplitude.

The experimental results are shown in Fig. 12 (b), (c) and (d). We have the following observations. (1) GenIA's performances increase with ω. Specifically, when ω increases from 0% to 50%, the performance improvement of GenIA is more obvious. When ω increases from 50% to 100%, the performance improvement is less significant. It indicates that augmenting 50% training samples is sufficient. (2) Template-level workload perturbation on queries instead causes a degradation in GenIA's performance. In particular, the extent of performance degradation gradually increases as ω rises. This is because excessive query perturbation may alter the index labels corresponding to the workload, thereby degrading the quality of the training data and ultimately leading to a decline in GenIA's performance. (3) For the parameter of attention perturbation α, when α is relatively small, the increase of α significantly improves the generalization ability of GenIA. However, a large α will overly sparsify the model parameters and result in performance degradation. The optimal value is $\alpha = 0.1$. (4) A similar trend is observed for the standard deviation of Gaussian Noise β. When β is small, increasing β enhances the generalization ability of GenIA. When β becomes too large, the learning of the model is severely distorted and results in performance degradation. The optimal value is $\beta = 0.1$.

J. Ablation on Featurization

There are three critical parts of the feature matrix, i.e., the plan features, the auxiliary features, and the data distribution features. To answer **Q3.2**, we design five variants of feature representation by replacing the critical parts. (1) **Column**: workload representation only reflects the appearance of indexable columns in each query, which is the feature representation adopted in [7]. (2) **w/o plan**: keeping the other parts unchanged and using the frequency of each query and the occurrence of indexable columns in each query to replace the plan features. (3) **w/o histogram**: keeping the other parts unchanged and discarding the data distribution features derived from the data histogram. (4) **w/o IUD**: keeping the other parts unchanged and discarding Equation. 4 the IUD statement features.

As shown in Fig. 13, (1) GenIA's feature mechanism significantly improves throughout and latency by 78.6% on average, compared with the **Column** feature representation used in other IAs [7]. (2) Comparing the effect of three critical parts, replacing the plan features with query content features causes the greatest damage. The variant **w/o plan** decreases GenIA's performance by 54.8%, indicating the importance of extracting workload features from query plans. (3) The variant **w/o IUD** reduces GenIA's performance by 25.7%, supporting our assumptions that data manipulation statements have a strong impact on index recommendation in Hybrid Transactional and

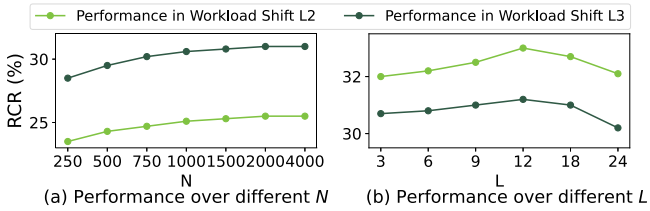

Fig. 14. Performance w.r.t. learning parameters.

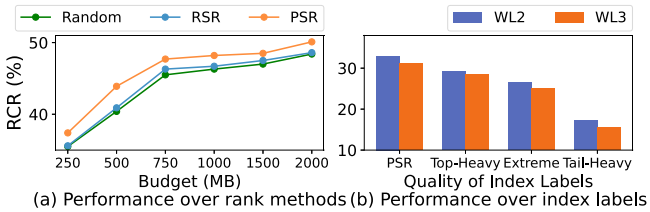

Fig. 15. RCR w.r.t. index labels.

Analytical Processing scenarios. (4) The performance of **w/o histogram** is 7.6% lower than GenIA's, showing that data distribution features can not be discarded.

K. Ablation on the Learning Parameters

In this section, we evaluate the impacts of various parameters in the learning process of GenIA, including the size of the training set (i.e., N) and the number of transformer blocks (i.e., L) in GenIA. We measure the *Relative Cost Reduction (RCR)* on TPC-H 1GB under workload and data shifts **WL2+DL1** and **WL3+DL1**, and vary $N = 500$, 750, 1000, 1500, 2000, 2500 and 4000, vary $L = 3$, 6, 9, 12, 18 and 24.

The experimental results are shown in Fig. 14. We have the following observations. (1) The performance of GenIA improves with more training instances. However, $N > 2000$ does not yield notable performance improvement. Thus, we set $N = 2000$, which is an affordable training cost. (2) For the parameter of the number of layers L. when L increases from 3 to 12, the performance improvement of GenIA is obvious. However, a larger L will lead to overfitting and degrade performance. Therefore, $L = 12$ is a suitable value.

L. Impact of Index Labels

In this section, we investigate the impacts of index labels on the TPC-H 1 GB, including the order of index labels in the target sequence and the quality of index labels. We use the *Relative Cost Reduction (RCR)* as the evaluation metric.

Firstly, under **WL1+DL1**, we conduct several variants regarding the output sequence when constructing the training samples. (1) **Random**. We use the original output by Extend, which can be considered a random order of indexes. (2) **RCR**. We sort indexes in descending order according to their *Relative Cost Reduction* on the corresponding workload. (3) **PSR**. We sort the indexes in descending order according to the *performance/size ratio* described in Section II-B.

The experimental results are shown in Fig. 15(a). We observe that: (1) PSR is significantly better than Random and RCR. Since indexes are built to meet a limited storage budget, PSR helps GenIA better learn how to select good indexes within the storage budget. (2) RCR's performance is slightly better than that of Random but much lower than that of PSR. This is because RCR ranks indexes solely by their benefits and ignores storage consumption. If an index brings a significant relative cost reduction but exceeds the storage budget, it is an ineffective index and should not be prioritized. In this sense, RCR does not cause greater benefits than random sorting.

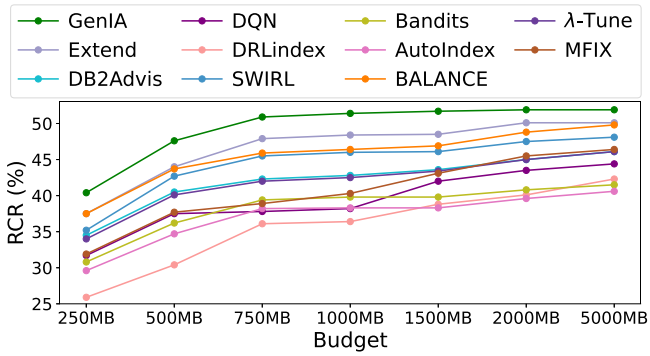

Fig. 16. RCR w.r.t. different storage budgets.

Secondly, we designed four variants of index labels under **WL2+DL1** and **WL3+DL1**: (1) PSR, which uses the complete sorted index sequence, representing the best-quality labels; (2) Top-Heavy, which retain the top two-thirds of the sorted index sequence (i.e., removes the last third). This variant emphasizes the most beneficial indices, representing a sub-optimal but still relatively strong set of labels. (3) Extreme, which retains the first third and the last third of the sequence, i.e., removes the middle third. This variant captures both highly beneficial and least beneficial indexes, representing a mixed-quality set of labels that stresses extremes rather than the middle ground. (4) Tail-Heavy, which retains the bottom two-thirds of the sequence (i.e., removes the first third). This variant emphasizes less beneficial indexes, representing the weakest-quality labels and testing robustness under poor index choices.

The experimental results shown in Fig. 15(b) indicate that: As the quality of index labels gradually degrades, the performance of GenIA also declines. This demonstrates that the quality of index labels significantly impacts GenIA's performance.

M. Impact of Storage Budget

In this section, we evaluate the impact of the storage budget on IAs' performance. We conduct experiments on TPC-H 1 GB using different storage budgets (i.e., 250 MB, 500 MB, 750 MB, 1000 MB, 1500 MB, 2000 MB, and 5000 MB) under **WL2+DL1**.

We observe the following conclusions from Fig. 16. (1) As the storage budget increases, the relative cost reduction brought by IAs' advised indexes increases significantly because the larger

storage budget allows the IAs to adopt better (and possibly more storage-consuming) indexes. (2) GenIA consistently shows optimal performance and surpasses the SOTA method Extend for all storage budgets, indicating that a well-trained GenIA is efficient (ie, with the 1% inference cost of Extend) and effective for various storage budgets. (3) When the storage budget changes from small to medium (e.g., from 250 MB to 750 MB), the relative cost reduction brought by the IAs' advised index increases significantly. In contrast, when the storage budget changes from medium to large (e.g., from 2000 MB to 5000 MB), the improvement is trivial. The underlying reason is that when the storage budget is limited, it is more important to apply index creation knowledge and focus on cost-effective indexes highly rewarded at unit storage cost. Nonetheless, there are only a few highly rewarding indexes, and a loose storage budget allows for greater flexibility in index selection.

VI. Related Work

To reduce labor costs in manual index tuning, many *index advisors* have been proposed. Existing index advisors can be divided into two categories: *heuristic-based* and *learning-based*.

A. Heuristic-Based Index Advisors

Heuristic-based IAs [4], [5], [14], [34], [35], [36], [37] use greedy algorithms to search index configurations. They typically analyze the workload patterns and filter the possible index candidates, including single-column index [36] and multi-column index [4], [5], [34], [35]). Then, they iteratively add an index to [4], [14], [34], [35], [36] or delete an index from [5], [37] the index configuration by predefined heuristics. Under the shifts of workload and data, heuristic-based algorithms always need to re-run from scratch because the index candidates have changed, and the heuristics to evaluate a candidate's "goodness" have also been altered. Thus, heuristic-based IAs have a large operating overhead in the dynamic scenario.

B. Learning-Based Index Advisors

Most learning-based index advisors adopt a reinforcement learning (RL) framework [38] that trains an index selection agent, i.e., a neural network [7], [8], [9], [10], [26], by interacting with the database environment. Several factors impede the effectiveness and efficiency of RL methods. (1) Generally, these methods prune a set of index candidates as the *action* space. Since the pruning is specific to the workload, the learned indexing policy is only effective within the action space explored during training and is unstable on different workloads. (2) Workload features are considered a part of the *state* of the database environment. Mostly, workload features are roughly defined, e.g., DQN [7] uses the frequency of queries. Thus, they assume the workloads are fixed and can not make reasonable predictions with different workloads. Recent methods [39] find that elaborate workload features improve the IA's robustness to workload shifts. Only DRLindex [8], [9], SWIRL [10] and BALANCE [26] with fine-grained workload features are competitive on dynamic workloads with unseen queries, but their workloads are limited to analytical queries. (3) The database data, where the workloads are carried out, has been largely overlooked in the state representation. Thus, the index predictions on dynamic data are unreliable.

Other trial-and-error methods include: AutoIndex [6], which chooses the best index candidates by simulating a Monte Carlo tree search; Bandits [1], which treats the index selection problem as a multi-arm bandit problem; MFIX [28], which sequentially refines its selection of index candidates through Bayesian Optimization. These methods also suffer from the stochastic nature of trials. Their predictions depend heavily on the trials with a fixed workload and data, and cannot achieve satisfactory performance on dynamic workloads and data.

Recently, with the rise of large models, several works (e.g., λ-Tune [27]), have attempted to utilize LLMs for database tuning, including index recommendation. Although λ-Tune enables cross-schema index recommendation without training, as shown in Section V-C, it incurs significant overhead due to multiple invocations with LLMs and extensive interaction with the database. Furthermore, LLM-based approaches appear inadequate for effectively addressing the index recommendation problem.

So far, only a few IAs have adopted a supervised learning paradigm. IdxL adopts two versions of supervised models. IdxL-c [13] trains separate classification models to predict different index candidates, e.g., one classifier for the single-column index, one for the two-column index, and so on. The ground-truth labels are extremely imbalanced, and the decision boundary is biased against classifying an index candidate as positive. IdxL-s [13] is the only generative IA in the literature. It fine-tunes the language model T5 [40], which is far more expensive (in training and inferring) than GenIA. Note that both versions of IdxL are only applied to slow queries and are infeasible for workloads because a workload consists of different queries and frequencies, and they also do not support setting a budget.

Summary: Existing IAs face the efficiency-effectiveness trade-off problem under workload and data shifts. To preserve index effectiveness on dynamic workloads and dynamic data, *heuristic-based* IAs need to re-execute their algorithms, most *RL-based* IAs need to re-train the agent, and other *trial-and-error* IAs need to update the sample trials. Thus, a large tuning overhead is inescapable and causes inefficiency. Furthermore, with the exception of λ-Tune, all other IAs are incapable of performing index recommendation across different database schemas.

VII. Conclusion

This paper studies the index selection problem in dynamic scenarios, especially how a learning-based IA can handle workload and data shifts. It demonstrates the potential of GenIA, a generative model in index recommendation, producing direct predictions without retraining or requiring massive positive and negative samples. GenIA ensures both the effectiveness and efficiency of index selection. The experiments define and explore a broader spectrum of workload and data shifts at different levels. Experimental results show that GenIA always surpasses the

near-optimal heuristic-based IA. i.e., Extend, with less than 1% inference cost, and the performance is stable under workload and data shifts. The innovations in the model, features, and training methods proposed in this paper are validated through ablation experiments. While `GenIA` currently assumes a fixed database schema to optimize recommendation quality for specific environments, extending its capability to generalize across schemas remains a promising direction for future research.

REFERENCES

[1] R. M. Perera, B. Oetomo, B. I. Rubinstein, and R. Borovica-Gajic, "No DBA? No regret! multi-armed bandits for index tuning of analytical and HTAP workloads with provable guarantees," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12855–12872, Dec. 2023.

[2] L. Ma, D. Van Aken, A. Hefny, G. Mezerhane, A. Pavlo, and G. J. Gordon, "Query-based workload forecasting for self-driving database management systems," in *Proc. ACM SIGMODc*, 2018, pp. 631–645.

[3] R. Cole et al., "The mixed workload CH-benCHmark," in *Proc. 4th Int. Workshop Testing Database Syst.*, 2011, pp. 1–6.

[4] R. Schlosser, J. Kossmann, and M. Boissier, "Efficient scalable multi-attribute index selection using recursive strategies," in *Proc. IEEE 35th Int. Conf. Data Eng.*, 2019, pp. 1238–1249.

[5] G. Valentin, M. Zuliani, D. C. Zilio, G. Lohman, and A. Skelley, "DB2 advisor: An optimizer smart enough to recommend its own indexes," in *Proc. 16th Int. Conf. Data Eng.*, 2000, pp. 101–110.

[6] X. Zhou et al., "AutoIndex: An incremental index management system for dynamic workloads," in *Proc. IEEE 38th Int. Conf. Data Eng.*, 2022, pp. 2196–2208.

[7] H. Lan, Z. Bao, and Y. Peng, "An index advisor using deep reinforcement learning," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, 2020, pp. 2105–2108.

[8] Z. Sadri, L. Gruenwald, and E. Lead, "DRLindex: Deep reinforcement learning index advisor for a cluster database," in *Proc. 24th Symp. Int. Database Eng. Appl.*, 2020, pp. 1–8.

[9] Z. Sadri, L. Gruenwald, and E. Leal, "Online index selection using deep reinforcement learning for a cluster database," in *Proc. IEEE 36th Int. Conf. Data Eng. Workshops*, 2020, pp. 158–161.

[10] J. Kossmann, A. Kastius, and R. Schlosser, "SWIRL: Selection of workload-aware indexes using reinforcement learning," in *Proc. Int. Conf. Extending Database Technol.*, 2022, pp. 2:155–2:168.

[11] W. Zhou, C. Lin, X. Zhou, and G. Li, "Breaking it down: An in-depth study of index advisors," *Proc. VLDB Endowment*, vol. 17, no. 10, pp. 2405–2418, 2024.

[12] Y. Zheng, C. Lin, X. Lyu, X. Zhou, G. Li, and T. Wang, "Robustness of updatable learning-based index advisors against poisoning attack," *Proc. ACM Manage. Data*, vol. 2, no. 1, pp. 1–26, 2024.

[13] G. Peng et al., "Online index recommendation for slow queries," in *Proc. IEEE 40th Int. Conf. Data Eng.*, 2024, pp. 5294–5306.

[14] N. Bruno and S. Chaudhuri, "Automatic physical database tuning: A relaxation-based approach," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2005, pp. 227–238.

[15] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[16] M. Zaheer et al., "Big bird: Transformers for longer sequences," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 17283–17297. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/c8512d142a2d849725f31a9a7a361ab9-Paper.pdf

[17] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," 2019, *arXiv:1904.10509*.

[18] S. Jaszczur et al., "Sparse is enough in scaling transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 9895–9907.

[19] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning deep features for discriminative localization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2921–2929.

[20] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le, "AutoAugment: Learning augmentation policies from data," 2018, *arXiv:1805.09501*.

[21] M. Poess and C. Floyd, "New TPC benchmarks for decision support and web commerce," *ACM SIGMOD Rec.*, vol. 29, no. 4, pp. 64–71, 2000.

[22] R. O. Nambiar and M. Poess, "The making of TPC-DS," in *Proc. Int. Conf. Very Large Data Bases*, 2006, pp. 1049–1058.

[23] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?," *Proc. VLDB Endowment*, vol. 9, no. 3, pp. 204–215, 2015.

[24] D. E. Difallah, A. Pavlo, C. Curino, and P. Cudre-Mauroux, "OLTP-bench: An extensible testbed for benchmarking relational databases," *Proc. VLDB Endowment*, vol. 7, no. 4, pp. 277–288, Dec. 2013.

[25] TPC, "TPC-C Benchmark," 2022. Accessed: 2022. [Online]. Available: http://www.tpc.org/tpcc/

[26] Z. Wang, H. Liu, C. Lin, Z. Bao, G. Li, and T. Wang, "Leveraging dynamic and heterogeneous workload knowledge to boost the performance of index advisors," *Proc. VLDB Endowment*, vol. 17, no. 7, pp. 1642–1654, 2024.

[27] V. Giannakouris and I. Trummer, "λ-Tune: Harnessing large language models for automated database system tuning," *Proc. ACM Manage. Data*, vol. 3, no. 1, pp. 1–26, 2025.

[28] Z. Chang, X. Zhang, Y. Li, X. Miao, Y. Qin, and B. Cui, "MFIX: An efficient and reliable index advisor via multi-fidelity Bayesian optimization," in *Proc. IEEE 40th Int. Conf. Data Eng.*, 2024, pp. 4343–4356.

[29] psycopg2, "psycopg2," 2013. Accessed: Jul. 23, 2024. [Online]. Available: https://github.com/psycopg/psycopg2

[30] HypoPG, "Hypopg," 2015. Accessed: Jul. 23, 2024. [Online]. Available: https://github.com/HypoPG/hypopg

[31] PostgreSQL, "Postgresql," 1996. Accessed: Jul. 23, 2024. [Online]. Available: https://www.postgresql.org/

[32] L. Zhang, C. Chai, X. Zhou, and G. Li, "LearnedSQLGen: Constraint-aware SQL generation using reinforcement learning," in *Proc. 2022 Int. Conf. Manage. Data*, 2022, pp. 945–958.

[33] X. Zhou et al., "FEBench: A benchmark for real-time relational data feature extraction," *Proc. VLDB Endowment*, vol. 16, no. 12, pp. 3597–3609, 2023.

[34] N. Bruno and S. Chaudhuri, "An online approach to physical design tuning," in *Proc. IEEE 23rd Int. Conf. Data Eng.*, 2006, pp. 826–835.

[35] S. Chaudhuri and V. R. Narasayya, "An efficient, cost-driven index selection tool for Microsoft SQL server," in *Proc. Int. Conf. Very Large Data Bases*, San Francisco, 1997, pp. 146–155.

[36] M. Hammer and A. Chan, "Index selection in a self-adaptive data base management system," in *Proc. 1976 ACM SIGMOD Int. Conf. Manage. Data*, 1976, pp. 1–8.

[37] K.-Y. Whang, "Index selection in relational databases," in *Foundations of Data Organization*, Berlin, Germany: Springer, 1987, pp. 487–500.

[38] V. Sharma and C. Dyreson, "Indexer workload-aware online index tuning with transformers and reinforcement learning," in *Proc. 37th ACM/SIGAPP Symp. Appl. Comput.*, 2022, pp. 372–380.

[39] W. Zhou, C. Lin, X. Zhou, G. Li, and T. Wang, "TRAP: Tailored robustness assessment for index advisors via adversarial perturbation," in *Proc. IEEE Int. Conf. Data Eng.*, 2024, pp. 42–55.

[40] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.

Xian Lyu is currently working toward the master's degree with the School of Informatics, Xiamen University, Xiamen, China. His research interests include generative AI for data systems, database index recommendation, and dynamic workload optimization, focusing on applying generative AI to data system optimization and designing efficient index recommendation strategies for dynamic workloads. He is actively engaged in data engineering research, participating in relevant projects and collaborating on database management system-related studies.

Chen Lin (Member, IEEE) received the BEng and PhD degrees from Fudan University, Shanghai, China, in 2004 and 2010, respectively. She is currently a professor with the School of Informatics and the National Institute for Data Science in Health and Medicine, Xiamen University, Xiamen, China. She has authored or coauthored more than 100 papers in international journals and top computer science conferences (including NeurIPS, ICML, KDD, WWW, SIGIR, SIGMOD, VLDB, and ICDE). Her research interests include profitable, unbiased, relevant, and efficient information filtering systems.

Yihang Zheng is currently working toward the master's degree with the School of Informatics, Xiamen University, Xiamen, China. His research interests include database systems, dynamic data workload adaptation, and AI-driven data management, focusing on addressing dynamic workload adaptation challenges and exploring AI-enabled efficient data management methods. He has also participated in research on the robustness of learning-based index advisors and related data management topics, advancing the integration of AI and database technologies.

Zhifeng Bao is currently a professor with the School of EECS, The University of Queensland, Brisbane, Australia. He has authored or coauthored in top databases and data mining conferences/journals. His research research interests include data management for AI, and AI-enhanced database systems. He was an associate editor for PVLDB and SIGMOD.

Yiming Zhang is currently a distinguished professor with Shanghai Jiao Tong University. He is also the director with NICE Lab (Laboratory of Networked Intelligent Computing at Exascale). He has authored or coauthored more than 50 highly influential papers in top conferences like NSDI, FAST, and EuroSys, and famous journals like TOCS, TOS, and TON, as the first/corresponding author. His research interests include operating, networking, storage, and AI systems. He is also an associate editor for *IEEE Transactions on Computers* and *IEEE Transactions on Services Computing*, and a member of the 2022 EuroSys Roger Needham Award Committee. He was the recipient of the Best Paper Award of USENIX FAST'23.

Guoliang Li (Fellow, IEEE) is currently a tenured full professor with the Department of Computer Science and Technology, Tsinghua University, Beijing, China. He has authored or coauthored more than 200 papers in top database conferences/journals. His research interests include data engineering, artificial intelligence for databases, human-in-the-loop data management, and large-scale data cleaning and integration, addressing key challenges in massive multi-source heterogeneous data processing and fusion. He is also an associate editor for *IEEE Transactions on Knowledge and Data Engineering* (TKDE).